

ROiCORE คืออะไร?

“ระบบรับรายงานน้ำท่วม ประเมินความเสี่ยง และแสดงผลบนแผนที่”

1. รับรายงานด่วน

ประชาชนรายงานเหตุน้ำท่วมได้ภายใน **≤ 3 Taps** พร้อมระบบระบุพิกัด GPS อัตโนมัติและแนบภาพถ่าย

2. ประเมินความเสี่ยง

คำนวณคะแนนความเสี่ยงอัตโนมัติ (น้ำขัง × พยากรณ์ฝน × ความถี่รายงาน) พร้อมวาดอาณาเขต **Polygon** เตือนภัย

3. แสดงผลบนแผนที่

แผนที่ OpenStreetMap แบบเรียลไทม์ ผสานภาพถ่ายดาวเทียม **GISTDA 77 จังหวัด** และแดชบอร์ดสรุปสถิติ

💬💡 พูดแค่นี้: “ROiCORE คือระบบรับรายงานสถานการณ์น้ำท่วม นำมาประเมินความเสี่ยงอัตโนมัติ และแสดงผลบนแผนที่แบบเรียลไทม์เพื่อช่วยเหลือประชาชนครับ”

🧩 Object ในระบบมีอะไรบ้าง?

การจำลองโลกความจริงของภัยพิบัติให้อยู่ในรูปแบบเชิงวัตถุ (Core Domain Entities & Value Objects)

📍 FloodPoint

ENTITY

จุดเฝ้าระวังน้ำท่วมบนแผนที่ ประกอบด้วย พิกัด, รัศมีผลกระทบ, ระดับน้ำ (เมตร) และ จำนวนรายงาน

📄 FloodReport

ENTITY

รายงานจากประชาชน ประกอบด้วย ระดับ ความรุนแรง (1-4), เบอร์โทรศัพท์, รูปถ่าย สถานที่ และเวลาที่เกิดเหตุ

📍 GeoLocation

VALUE OBJECT

วัตถุพิกัดภูมิศาสตร์ (Lat / Lng) ควบคุม ความถูกต้องของพิกัดในประเทศไทย (Lat -90 ถึง 90, Lng -180 ถึง 180)

⚠️ RiskLevel

ENUM

ระดับความเสี่ยง 4 ระดับ: **CRITICAL** (วิกฤต), **HIGH** (สูง), **MEDIUM** (ปานกลาง), **LOW** (เฝ้าระวัง)

💬 พุดแค่นี้: "ในระบบเราแบ่ง Object หลักออกเป็น 4 ตัวคือ FloodPoint จุดน้ำท่วม, FloodReport รายงานประชาชน, GeoLocation ดูแลพิกัด และ RiskLevel กำหนดระดับความเสี่ยงครับ"



ออกแบบ OOP ยังไง? (Class Diagram)

แยกความรับผิดชอบอย่างชัดเจนตามหลัก Clean Architecture และ Dependency Inversion

Domain Entities

FloodPoint

- id: string
- location: GeoLocation
- waterLevel: float
- riskLevel: RiskLevel
- + evaluateRisk(strategy)

FloodReport

- severity: Severity
- phone: string
- photoUrl: ?string

Strategies & Interfaces

```
<<interface>> IRiskStrategy
+ calculate(point): RiskLevel
↳ CompositeRiskStrategy
```

```
<<interface>> IFloodRepository
+ save(report): void
+ findActivePoints(): array
```

Repositories & Adapters

SupabaseFloodPointRepo

↳ เชื่อมต่อ Postgres / PostGIS

GistdaDisasterService

↳ ดึงภาพดาวเทียม 77 จังหวัด

OpenWeatherService

↳ สภาพอากาศและพยากรณ์ฝน



พูดแค่นี้: “เราออกแบบ OOP โดยแยก Domain ออกจาก Database ชัดเจน ทุก Service คุยกันผ่าน Interface ทำให้ทดสอบง่ายและเปลี่ยน Database ได้โดยไม่กระทบ Business Logic ครับ”

3 Design Patterns หัวใจสำคัญ

แก้ปัญหาความซับซ้อนของระบบด้วย GoF Patterns ที่ตรงจุดและมีประสิทธิภาพ

Strategy Pattern

“คำนวณความเสี่ยง”

CompositeRiskStrategy: คำนวณความเสี่ยง
อัตโนมัติจาก (ระดับน้ำท่วม × พยากรณ์ฝน 3 ชม. ×
ความหนาแน่นของรายงาน) สามารถสลับสูตรได้อิสระ

Repository Pattern

“จัดการข้อมูล”

SupabaseFloodReportRepository: ห่อหุ้มการ
Query ฐานข้อมูล แยกคำสั่ง SQL/PostGIS ออกจาก
โค้ดระบบ ทำให้สามารถสลับไปใช้ Mock ในการทดสอบได้
ทันที

Adapter Pattern

“เชื่อมต่อ API ภายนอก”

**GistdaDisasterService &
OpenWeatherService:** แปลงโครงสร้างข้อมูล
ดาวเทียม GISTDA และสภาพอากาศภายนอกให้อยู่ใน
มาตรฐานเดียวกันของระบบ

💬💡 พุดแค่นี้: “เราใช้ 3 Pattern หลัก: Strategy ใช้คำนวณความเสี่ยง, Repository ใช้จัดการฐานข้อมูล, และ Adapter ใช้แปลงข้อมูลจาก API ภายนอกอย่าง GISTDA ครับ”

🎬 ระบบทำงานยังไง? (4 ขั้นตอน)

การเดินทางของข้อมูลตั้งแต่ประชาชนกรดยางาน จนถึงการวาดอาณาเขตบนแผนที่สด

01

รายงาน (Report)

ประชาชนเลือก 1 ใน 4 ระดับน้ำ ผ่านฟอร์ม
ผ่าน 3 Taps ระบบดึงพิกัด GPS อัตโนมัติ

02

ประเมิน (Evaluate)

Strategy วิเคราะห์ระดับความรุนแรง และ
ตรวจจับ Cluster หากมีการรายงานซ้ำซ้อน

03

บันทึก (Persist)

Repository บันทึกลง Supabase
Postgres + PostGIS และแคชข้อมูลเพื่อ
ความเร็ว

04

แสดงผล (Render)

โหลดไฟล์บน OpenStreetMap ทับที่
พร้อมวาดอาณาเขต Polygon สีแดงเตือน
ภัย

💬💡 พูดแค่นี้: “วงจรการทำงานง่ายมากครับ: 1. รายงานเหตุ → 2. ระบบประเมินความเสี่ยง → 3. บันทึกข้อมูล → และ 4. แสดงหมุดพร้อมวาด Polygon บนแผนที่กันทีครับ”



Demo ระบบจริง + สรุปผลงาน

เปิดระบบทดสอบการทำงานจริงแบบเรียลไทม์ และสรุปคุณค่าที่ส่งมอบ

▶ ลำดับขั้นตอนการสาธิต (Demo Steps)

1. **เปิดระบบจริง:** เข้าสู่หน้าเว็บ <http://localhost:8000/>
2. **รายงานน้ำท่วมด่วน:** กดปุ่มสีน้ำเงิน → เลือก "รถเล็กผ่านไม่ได้" → กดยืนยัน
3. **ดูผลบน Map:** ดูหมุดสีส้ม/แดง โผล่ขึ้นมาบน OpenStreetMap กันที
4. **ตรวจสอบแดชบอร์ด:** สลับไปหน้า /dashboard ดูตัวเลข KPI อัปเดตสด

🏆 สรุปคุณค่าของ ROiCORE

- ✓ **สถาปัตยกรรม OOP แข็งแกร่ง:** ขยายต่อยอดและดูแลง่าย
- ✓ **White Modern UI:** ไร้ขอบ ไร้เงา สบายตา สดส่วนกะทัดรัด
- ✓ **พร้อมใช้งานจริง:** เชื่อมต่อ GISTDA 77 จังหวัด และใช้งานได้ทันที

💬 พูดแค่นี้: “ต่อไปมาชมการ Demo ระบบจริงกันครับ เราจะเปิดเว็บ ส่งรายงานด่วน แล้วดูผลลัพธ์บนแผนที่ Leaflet สดๆ กันทีครับ ขอขอบคุณครับ”